De documentos a solicitudes verificables

Laboratorio de RPA · Python en VS Code · 180 minutos

Propósito y entrega

Obtendrás id_solicitud, solicitante, area, tipo y descripcion desde documentos sintéticos. Compararás reglas y LLM usando exactamente el mismo texto OCR y las mismas validaciones. No se presupone experiencia previa con OCR o API.

Entrega mínima: tres textos OCR, tres JSON de reglas, tres JSON de LLM real o explícitamente simulado, reporte por campo, casos pendientes y reflexión. Si trabajas sin API, demuestra el circuito y explica que no evaluaste un modelo real. El CSV puede contener solo encabezados si no hay casos aprobados. PAD es una ampliación posterior.

Mapa del proceso

Original PNG/PDF escaneado -> renderizado completo del PDF -> OCR original conservado -> dos ramas: reglas / LLM -> mismo validador -> comparación con referencia -> revisión humana contra original -> CSV de un solo método -> PAD posteriormente.

El PDF digital sigue otra ruta: extracción directa de texto, sin OCR. En este laboratorio se usa como contraste; la comparación de métodos usa los escaneados.

Ritmo de la sesión

• 20 min: entorno y tipos de documento.

• 35 min: PDF e imágenes a texto OCR.

• 35 min: extracción mediante reglas.

• 45 min: extracción mediante LLM.

• 30 min: validación, comparación y revisión.

• 15 min: evidencia y cierre.

La instalación se completa antes de clase. Los tiempos incluyen inspección del código, análisis y ejercicios; ejecutar comandos no es el objetivo completo.

Convención de los pasos

Cada E indica dónde trabajar, qué hacer, qué adaptar, qué archivo esperar, cómo comprobarlo y qué hacer si falla. Los comandos se ejecutan desde la raíz del proyecto, la carpeta que contiene requirements.txt y scripts. Usa la variante de tu sistema, una sola vez.

Preparación previa · Python y VS Code

E01. Extraer y abrir el proyecto [manual]

En el explorador de archivos, extrae completamente el ZIP en una carpeta propia, por ejemplo C:\RPA\laboratorio_extraccion en Windows o ~/Cursos/laboratorio_extraccion en Mac. En VS Code selecciona Archivo -> Abrir carpeta y elige esa carpeta. Instala la extensión Python de Microsoft desde Extensiones si falta. Abre Terminal -> Nueva terminal.

Adapta únicamente la ubicación. Salida esperada: explorador con entradas, scripts y requirements.txt. Comprueba que la terminal está en la carpeta con requirements.txt; usa Get-Location en PowerShell o pwd en Mac. Si aparece una carpeta contenedora adicional, abre la carpeta interior.

E02. Crear el entorno [terminal]

Instala Python 3.12 si no lo tienes, desde python.org; en Windows incluye el lanzador py. En la terminal de VS Code ejecuta:

Windows PowerShell:

```
py -3.12 --version
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

macOS:

```
python3.12 --version
python3.12 -m venv .venv
./.venv/bin/python -m pip install -r requirements.txt
```

No adaptes los nombres de paquetes. Salida: carpeta .venv e instalación sin errores. Si py o python3.12 no existen, instala Python 3.12 y reabre VS Code. Si pip falla, conserva el mensaje y revisa conexión y compatibilidad antes de continuar. No necesitas activar el entorno ni cambiar la política de ejecución de Windows.

E03. Seleccionar y verificar el intérprete [VS Code y terminal]

Pulsa Ctrl+Shift+P en Windows o Cmd+Shift+P en Mac. Busca Python: Select Interpreter y elige el Python dentro de .venv. Si no aparece, utiliza Enter interpreter path y busca .venv\Scripts\python.exe o .venv/bin/python.

Windows:

```
.\.venv\Scripts\python.exe -c "import sys; print(sys.executable)"
```

Mac:

```
./.venv/bin/python -c "import sys; print(sys.executable)"
```

Salida esperada: ruta de .venv. Si apunta a otro Python, repite la selección. Los comandos explícitos de esta guía siguen usando el entorno aunque la terminal no esté activada.

Preparación previa · motor OCR

E04. Instalar Tesseract y español [manual y terminal]

pip instala pytesseract, que es un adaptador; no instala el motor Tesseract ni su idioma spa. En Windows sigue el enlace a los instaladores de UB Mannheim desde la documentación oficial de Tesseract indicada al final. Instala Tesseract 5 y selecciona español si el instalador lo ofrece. La ruta habitual es C:\Program Files\Tesseract-OCR.

Windows PowerShell, adapta la ruta si cambió:

```
& 'C:\Program Files\Tesseract-OCR\tesseract.exe' --version
& 'C:\Program Files\Tesseract-OCR\tesseract.exe' --list-langs
```

Si falta spa, descarga spa.traineddata del repositorio oficial tessdata_fast y colócalo en la carpeta tessdata del motor instalado. Si el equipo institucional restringe escritura, pide la instalación al responsable; no cambies las políticas del equipo.

En macOS, con Homebrew instalado según brew.sh:

```
brew install tesseract tesseract-lang
tesseract --version
tesseract --list-langs
```

Salida esperada: versión del motor y una lista que contiene spa. Si el comando no existe, reabre la terminal o utiliza su ruta completa. Si falta spa, revisa el paquete de idioma y la carpeta tessdata. No continúes con OCR en español hasta que aparezca.

E05. Preparar configuración sin compartir secretos [manual]

En VS Code copia .env.example como .env en la raíz. No hace falta una clave para OCR ni simulación. En Windows quita el comentario de TESSERACT_CMD y deja la ruta real usando barras /:

```
TESSERACT_CMD=C:/Program Files/Tesseract-OCR/tesseract.exe
```

En Mac normalmente se localiza por PATH; si falla, usa /opt/homebrew/bin/tesseract en Apple Silicon o la ruta que devuelva command -v tesseract. TESSDATA_PREFIX es opcional y debe apuntar a la carpeta que contiene spa.traineddata si el motor no la encuentra.

Salida: .env local, excluido de Git. Comprueba su nombre exacto: no debe ser .env.txt. No fotografíes ni publiques su contenido. Si se usa API, el docente configurará la clave localmente más adelante; no la pegues en chats.

Bloque 1 · ¿Qué documento tienes?

E06. Inspeccionar los originales [análisis manual]

Abre entradas/C01.png y C01.pdf: solicitud con etiquetas. Abre C02.png: misma finalidad en párrafo. Abre C03.png: documento degradado y un campo ausente. Abre C01_digital.pdf e intenta seleccionar texto. No modifiques los originales.

Adapta nada. Salida: anota en PLANTILLA_RESULTADOS.md qué formatos tienen texto seleccionable y qué campos ves. Comprueba que el campo Tipo no está en C03; no lo deduzcas de la descripción. Si no abre un PDF, usa el visor del sistema.

E07. Ejecutar inspección directa [terminal]

Windows:

```
.\.venv\Scripts\python.exe scripts/E01_inspeccionar_documento.py
```

Mac:

```
./.venv/bin/python scripts/E01_inspeccionar_documento.py
```

Salida: salidas/inspeccion.json y salidas/directo/*.txt. Comprueba caracteres: los tres PDF escaneados deben tener cero; el digital debe tener texto. Si el escaneado tiene texto, conserva el hallazgo y pide al docente revisar el paquete. No lo llames OCR: Page.get_text extrae una capa ya existente.

E08. Explicar el alcance [análisis]

En la plantilla explica: una imagen tiene píxeles; el OCR intenta convertirlos en texto. Extraer campos organiza ese texto, y validar comprueba reglas. Ninguna de estas operaciones demuestra por sí sola que un dato coincida con el original.

Salida: cuatro definiciones y un ejemplo de error por etapa. Comprueba que no atribuyes al LLM acceso al original: recibirá solo el texto OCR. Si confundes las etapas, vuelve al mapa de la primera página y señala la entrada y salida de cada una.

Bloque 2 · OCR conservado

E09. Renderizar y reconocer [terminal]

Windows:

```
.\.venv\Scripts\python.exe scripts/E02_extraer_texto_ocr.py
```

Mac:

```
./.venv/bin/python scripts/E02_extraer_texto_ocr.py
```

No adaptes parámetros en el primer ensayo. El script usa PDF, renderiza páginas completas a 300 dpi y aplica spa con --psm 6. Salidas: salidas/ocr/C01.txt a C03.txt, metadatos JSON y PNG por página. Comprueba que existen los tres textos y sus imágenes renderizadas. Si falta el motor o spa, regresa a E04-E05. Si falla a mitad, conserva y renombra la carpeta ocr antes de repetir; el script impide sobrescribirla.

E10. Comparar texto y documento [VS Code y visor]

Abre cada .txt junto a su PNG original. Copia en la plantilla al menos tres observaciones, incluyendo un acento, un identificador y el campo ausente. No corrijas ni sobrescribas el OCR. Un texto legible también puede contener errores silenciosos.

Salida: observaciones por caso. Comprueba especialmente SOL-801 a SOL-803. Si la degradación no provoca errores en tu motor, informa ese resultado: degradar una imagen no garantiza fallos.

E11. Inspeccionar el renderizado [código y ejercicio]

Abre scripts/E02_extraer_texto_ocr.py. Localiza pagina.get_pixmap(dpi=300, alpha=False). Explica por qué renderizar la página completa conserva el diseño mientras extraer imágenes incrustadas puede perder textos, giros o capas.

Opcional, para probar PNG directamente sin sustituir el ensayo principal:

Windows:

```
.\.venv\Scripts\python.exe scripts/E02_extraer_texto_ocr.py --formato png --ensayo png
```

Mac:

```
./.venv/bin/python scripts/E02_extraer_texto_ocr.py --formato png --ensayo png
```

Salida: salidas/ocr_png. Compara con ocr; E03 y E04 seguirán leyendo ocr. Si ya existe el ensayo, renómbralo antes de repetir. No mezcles ambos textos en una comparación.

Bloque 3 · Extraer sin LLM

E12. Leer las reglas [código]

Abre scripts/comun.py y busca extraer_reglas. La expresión ^ identifica comienzo de línea; \s* permite espacios; las etiquetas conocidas identifican campos; la búsqueda termina ante la siguiente etiqueta o el final. re.M trabaja por líneas y re.S permite valores que ocupen varias líneas.

Los nombres de personas y descripciones no están codificados como respuestas. El patrón SOL detecta un identificador único en texto libre; los demás campos requieren etiquetas. Un resultado null indica que no se encontró información de manera segura, no una cadena vacía inventada.

Salida: explica en la plantilla por qué C02 puede fallar. Si no reconoces el patrón, identifica primero una sola etiqueta en C01.txt y sigue su valor hasta la siguiente.

E13. Ejecutar reglas [terminal]

Windows:

```
.\.venv\Scripts\python.exe scripts/E03_extraer_con_reglas.py
```

Mac:

```
./.venv/bin/python scripts/E03_extraer_con_reglas.py
```

Salida: salidas/reglas/C01.json a C03.json. Comprueba datos y ocr_sha256. Espera más campos ausentes en el párrafo que en el formato fijo; registra lo observado. Si falta un .txt, completa E09. Si el JSON es válido pero un valor incorrecto, es un problema de extracción que debes documentar, no ocultar.

E14. Modificar y comprobar una regla [ejercicio]

Antes de cambiar comun.py, guarda una copia fuera de scripts con otro nombre. Añade soporte para la etiqueta Área escrita como Area: modifica tanto el patrón de etiquetas como el mapa de equivalencias. No agregues nombres, IDs o descripciones de los casos al código.

Para probar, usa una cadena nueva en una prueba dentro de pruebas/test_laboratorio.py con Area: Docencia. Ejecuta las pruebas indicadas en E24 y anota el antes/después. Salida: explicación de dos líneas cambiadas y resultado de la prueba. Si rompes una prueba, revisa patrón y mapa o restaura tu copia. Conserva los originales OCR.

Bloque 4 · LLM con estructura

E15. Inspeccionar contrato y prompt [código]

Abre Extraccion en comun.py: cinco campos de texto o null y observaciones. Todos deben existir; propiedades adicionales y tipos incorrectos se rechazan. Abre PROMPT en E04_extraer_con_llm.py. Identifica las instrucciones para tratar el documento como datos, abstenerse ante ambigüedad y no inventar campos.

Salida: explica qué garantiza un esquema y qué no. Comprueba que no existe un campo de confianza tomado como garantía. Si falta información, debe seguir siendo null aunque el formato pida una propiedad.

E16. Ejecutar sin API [terminal]

Windows:

```
.\.venv\Scripts\python.exe scripts/E04_extraer_con_llm.py --modo simulado
```

Mac:

```
./.venv/bin/python scripts/E04_extraer_con_llm.py --modo simulado
```

Salida: salidas/llm_simulado/*.json, con modo simulado. El simulador reutiliza reglas para ejercitar el circuito: no es un LLM local y no demuestra rendimiento de un modelo. No carga .env ni realiza red. Comprueba la observación SIMULADO y compara las huellas con reglas. Si falta OCR, vuelve a E09.

E17. Preparar API real, solo con el docente [manual]

El docente configura OPENAI_API_KEY dentro de .env local. No se solicita en chat, no se publica ni se comparte con el grupo. OPENAI_MODEL permite cambiar gpt-4.1-mini por otro modelo compatible con Responses y salidas estructuradas que la cuenta tenga disponible. Las llamadas tienen costo y la disponibilidad depende de la cuenta.

Salida: configuración local, sin capturas del secreto. Antes de llamar, el docente autoriza el gasto para tres documentos y comprueba su presupuesto. Si no hay autorización, clave o cuota, continúa con simulación y declara la limitación. No cambies de modelo por ensayo y error con llamadas pagadas.

Bloque 4 · Ejecución real y contingencias

E18. Llamar solo después de autorización [terminal]

Este paso realiza hasta tres llamadas pagadas. Es una actividad opcional bajo control docente. No lo ejecutes para comprobar si funciona una clave.

Windows:

```
.\.venv\Scripts\python.exe scripts/E04_extraer_con_llm.py --modo real --autorizar-costo
```

Mac:

```
./.venv/bin/python scripts/E04_extraer_con_llm.py --modo real --autorizar-costo
```

Salida: salidas/llm_real/*.json. Comprueba modo real, estado ok, modelo y datos. No se distribuyen estos resultados privados. Si hay un fallo técnico, el JSON conserva datos null y estado fallo_tecnico; no es una solicitud válida. Las llamadas usan timeout de 45 segundos y cero reintentos automáticos. Una interrupción puede dejar resultados parciales: revisa antes de volver a llamar.

E19. Diagnosticar sin exponer credenciales [análisis]

• autenticacion_fallida: docente revisa clave local; no la imprime.

• acceso_denegado: docente revisa permisos de proyecto y modelo.

• limite_o_cuota: revisa cuota/presupuesto y límites; no repitas automáticamente.

• conexion o tiempo_agotado: revisa red; una llamada interrumpida puede haber tenido costo.

• respuesta_incompleta: no uses un JSON parcial; revisa longitud y causa.

• rechazo_modelo: conserva el rechazo como fallo; no rellenes campos.

• respuesta_invalida o sin estructura: detén esa extracción y revisa el contrato.

Salida: anota categoría y contingencia en la plantilla, sin copiar mensajes que incluyan secretos. Si no se puede resolver durante clase, vuelve al modo simulado y separa sus conclusiones.

E20. Comparar la misma entrada [código y análisis]

Abre un JSON de reglas y el correspondiente de LLM. Sus ocr_sha256 y fuente_sha256 deben coincidir. Solo se envía texto OCR a la API: ni PNG, ni PDF, ni referencia. El conjunto de referencia se carga exclusivamente en E05.

Salida: anota ambas huellas y el modo del LLM. Si difieren, no compares esos resultados: conserva el ensayo y vuelve a extraer ambos desde la misma carpeta ocr. El LLM no garantiza corregir errores de reconocimiento y puede generar datos plausibles incorrectos.

Bloque 5 · Validar y comparar

E21. Generar el reporte [terminal]

Para trabajar con simulación y preparar la revisión del método reglas:

Windows:

```
.\.venv\Scripts\python.exe scripts/E05_validar_comparar_exportar.py --metodo reglas --llm-modo simulado
```

Mac:

```
./.venv/bin/python scripts/E05_validar_comparar_exportar.py --metodo reglas --llm-modo simulado
```

Si realizaste E18, cambia únicamente --llm-modo a real. Elige --metodo llm solo si quieres que ese único método alimente el CSV. No se fusionan ambas ramas. Salidas: REPORTE.md, comparacion.json, pendientes_revision.json, revision_humana.json, revision_plantilla_actual.json, exportacion.json y solicitudes_para_pad.csv en salidas.

Comprueba que el CSV inicialmente tiene solo encabezados: nadie ha aprobado los campos. Si el reporte indica archivo ausente, completa esa extracción. Si dice entrada modificada o distinta, conserva el ensayo y repite ambos métodos desde una misma fuente.

E22. Interpretar resultados [análisis]

Abre comparacion.json y referencia.json ahora, después de extraer. La referencia no se proporciona a E03 ni a E04. Se normalizan únicamente Unicode NFC y espacios; no se corrigen acentos, números, mayúsculas ni sinónimos. La descripción se compara literalmente tras esas normalizaciones; una paráfrasis puede expresar algo similar y aun así no coincidir.

Distingue estas categorías en la plantilla:

• Error OCR: original y texto difieren; confirma visualmente.

• Error de extracción: el texto tiene el dato, pero se extrajo mal.

• Incumplimiento de reglas: longitud, formato o catálogo inválido.

• Ausente o incierto: falta información o hay ambigüedad; conservar null.

• Fallo técnico: no hay resultado utilizable, por ejemplo cuota agotada.

El reporte automático marca posible_error_ocr cuando no localiza el valor esperado; es una hipótesis que puede deberse también a redacción o normalización. Confírmala con el original. Registra diagnósticos por campo en revision_humana.json; un caso puede tener más de una causa. La simulación no tiene puntuación de rendimiento del modelo.

Bloque 5 · Revisión y CSV

E23. Autorizar solo lo verificado [manual y terminal]

Abre salidas/revision_humana.json y el PDF o PNG original indicado en el JSON del resultado. Cada clave identifica caso y método, por ejemplo C01:reglas. Compara los cinco campos, uno por uno, con el original. Solo si todos son fieles, cambia sus cinco campos_verificados a true, escribe un revisor identificable y cambia aprobado a true. Conserva firma tal como la generó el script. Actualiza diagnostico y notas con lo que observaste.

No apruebes C03: falta Tipo. No inventes el dato. No apruebes resultados simulados ni campos incorrectos aunque cumplan formato. La firma vincula la declaración al resultado y al original; no sustituye la revisión ni es una firma digital de identidad.

Reejecuta exactamente el comando E21. Salida: CSV UTF-8 con BOM, separado por comas, con estas columnas exactas:

```
id_solicitud,solicitante,area,tipo,descripcion
```

Comprueba que solo contiene filas aprobadas del método seleccionado. Los IDs repetidos entre candidatos se excluyen todos. La aplicación PAD también rechazará un ID que ya exista en su historial; este laboratorio no consulta ese historial. Si el CSV sigue vacío, revisa formato, modo, revisor, cinco verificaciones y firma.

Si regeneraste un resultado, no copies una firma a ciegas: vuelve a comparar con el original y usa la entrada de revision_plantilla_actual.json para sustituir únicamente el caso revisado. Si un campo está mal, déjalo pendiente; no edites el resultado para simular aciertos. Cualquier corrección manual debe conservarse como una actividad posterior separada.

E24. Ejecutar pruebas [terminal]

Windows:

```
.\.venv\Scripts\python.exe -m unittest discover -s pruebas -v
```

Mac:

```
./.venv/bin/python -m unittest discover -s pruebas -v
```

Salida: pruebas finalizadas con OK. Comprueba casos ausentes, IDs incorrectos, duplicados, rechazos y respuestas inválidas. Si una prueba falla tras E14, revisa el cambio antes de entregar. Las pruebas no llaman a la API.

Bloque 6 · Evidencias y cierre

E25. Preparar la entrega [manual]

Completa PLANTILLA_RESULTADOS.md. Conserva los tres .txt y metadatos de OCR, los JSON de ambos métodos, REPORTE.md, comparacion.json, pendientes_revision.json y el CSV aunque solo tenga encabezado. Incluye el archivo de revisión que justifica las filas aprobadas. No entregues .env, la clave, .venv ni cachés.

Salida: carpeta de evidencia identificada por tu equipo y ensayo. Comprueba que cada afirmación indica real o simulado. Como salidas está excluida de Git, entrega sus evidencias por el canal del curso; no cambies el .gitignore para publicar una ejecución privada.

E26. Argumentar lo aprendido [análisis]

Redacta 150-200 palabras: ¿qué se pierde entre imagen y texto?, ¿en qué caso fallaron las reglas?, ¿qué observaste realmente del modelo o qué no pudiste medir?, ¿por qué un formato válido no autoriza captura? Propón una mejora general sin memorizar los casos.

Salida: reflexión y una tabla por campo/caso en la plantilla. Comprueba que no concluyes que LLM supera a reglas si solo ejecutaste el simulador. Si no hay API real, se evalúan el circuito, la lectura crítica y las limitaciones declaradas.

E27. Ampliación posterior con PAD [opcional]

Solo con CSV aprobado, adapta la ruta del flujo de solicitudes a salidas/solicitudes_para_pad.csv. Reutiliza captura por UI, confirmación, lectura del folio y bitácora del proyecto anterior. Comprueba que los IDs no existen ya. Esta captura no forma parte de los 180 minutos ni condiciona la entrega mínima.

Si algo no funciona

ModuleNotFoundError: ejecuta pip con el mismo Python explícito de E02. Ruta no encontrada: verifica la raíz y los nombres; los scripts resuelven sus datos con pathlib desde su propia ubicación. OCR vacío: abre el PNG renderizado, revisa orientación e idioma; conserva el texto original y crea otro ensayo para experimentar. JSON inválido al editar revisión: revisa comas, comillas dobles y true/false en minúsculas. Salidas antiguas: preserva la carpeta completa con otro nombre y repite la secuencia; no mezcles ensayos.

Referencias y decisiones técnicas

Fuentes oficiales consultadas el 13 de octubre de 2026.

Dependencias del proyecto

PyMuPDF proporciona extracción directa y renderizado real de páginas PDF en una misma biblioteca. Evita exigir Poppler al alumnado. PyMuPDF se distribuye bajo AGPL o licencia comercial; revisar su licencia antes de reutilizarlo fuera de este ejercicio académico.

https://pymupdf.readthedocs.io/en/latest/page.html

pytesseract es el adaptador Python para Tesseract; Pillow abre las imágenes. Tesseract es un motor externo y spa.traineddata es un recurso de idioma separado. La preparación del equipo debe comprobar ambos.

https://github.com/madmaze/pytesseract

https://tesseract-ocr.github.io/tessdoc/Installation.html

https://github.com/tesseract-ocr/tessdata_fast/blob/main/spa.traineddata

El SDK oficial OpenAI utiliza Responses API y responses.parse con un modelo Pydantic. Se configura gpt-4.1-mini porque su ficha documenta soporte de salidas estructuradas. No es una promesa de acceso, calidad ni precio fijo. Consulta disponibilidad y costos antes de autorizar una ejecución real.

https://developers.openai.com/api/docs/guides/structured-outputs

https://developers.openai.com/api/docs/models/gpt-4.1-mini

https://developers.openai.com/api/docs/pricing

Pydantic valida el contrato de salida; python-dotenv carga configuración local. csv, pathlib, hashlib, re y unittest pertenecen a la biblioteca estándar. ReportLab se usa solo para generar materiales docentes, no es una dependencia de ejecución del alumnado.

https://docs.pydantic.dev/latest/

https://github.com/theskumar/python-dotenv

https://docs.python.org/3/library/csv.html

https://code.visualstudio.com/docs/python/environments

Estado de validación

Consulta docente/VALIDACION.md en la versión docente para la evidencia de ejecución de esta preparación. En la distribución estudiantil consulta ESTADO_VALIDACION.md. Un ensayo portable en macOS no acredita Windows. La API real no se ejecuta durante la preparación sin autorización explícita.